

DSA Array Interview Cheat Sheet

JavaScript solutions · 24 problems · 7 patterns · time & space complexity

Array questions in interviews almost always reduce to one of a handful of patterns. Spot the pattern from the shape of the input and the rest of the solution tends to follow. Use the table below to recognize which one you're looking at, then jump to that section.

Pattern	Recognize it by	Typical time
Two Pointers	Sorted array, pair/triplet sums, palindromes, merging in place	$O(n)$ – $O(n \log n)$
Sliding Window / Kadane's	"Contiguous subarray", running max/min/sum	$O(n)$
Hashing / Prefix Sum	Need $O(1)$ lookups, "count pairs/subarrays that..."	$O(n)$
Sorting First	Output order doesn't matter, need grouping/comparison	$O(n \log n)$
Binary Search	Sorted or rotated-sorted array, "find index/boundary"	$O(\log n)$
Matrix Traversal	2D grid, rotate / spiral / zero-out, boundary tracking	$O(m \cdot n)$
Intervals	Overlapping ranges, scheduling, merging	$O(n \log n)$

1 Two Pointers

3Sum ● Medium

Sort the array, fix one element, then use two pointers on the rest to find pairs that sum to its negative. Skip duplicate values at every level to avoid repeat triplets.

```
function threeSum(nums) {
  nums.sort((a, b) => a - b);
  const result = [];
  for (let i = 0; i < nums.length - 2; i++) {
    if (i > 0 && nums[i] === nums[i - 1]) continue;
    let left = i + 1, right = nums.length - 1;
    while (left < right) {
      const sum = nums[i] + nums[left] + nums[right];
      if (sum === 0) {
        result.push([nums[i], nums[left], nums[right]]);
        while (left < right && nums[left] === nums[left + 1]) left++;
        while (left < right && nums[right] === nums[right - 1]) right--;
        left++;
        right--;
      } else if (sum < 0) {
        left++;
      } else {
        right--;
      }
    }
  }
  return result;
}
```

Time: $O(n^2)$ **Space:** $O(1)$ extra (excl. sort)

Container With Most Water ● Medium

Start with the widest container (pointers at both ends) and always move the shorter side inward — moving the taller side can only shrink the area.

```
function maxArea(height) {
  let left = 0, right = height.length - 1;
  let max = 0;
  while (left < right) {
    const area = Math.min(height[left], height[right]) * (right - left);
    max = Math.max(max, area);
    if (height[left] < height[right]) left++;
    else right--;
  }
  return max;
}
```

Time: $O(n)$ **Space:** $O(1)$

Trapping Rain Water ● Hard

Track the tallest wall seen so far from each side. Water trapped at a position is bounded by the smaller of the two max walls, so advance whichever side has the lower max.

```
function trap(height) {
  let left = 0, right = height.length - 1;
  let leftMax = 0, rightMax = 0;
  let water = 0;
  while (left < right) {
    if (height[left] < height[right]) {
      if (height[left] >= leftMax) {
        leftMax = height[left];
      } else {
        water += leftMax - height[left];
      }
      left++;
    } else {
      if (height[right] >= rightMax) {
        rightMax = height[right];
      } else {
        water += rightMax - height[right];
      }
      right--;
    }
  }
  return water;
}
```

Time: $O(n)$ **Space:** $O(1)$

Move Zeroes ● Easy

Maintain a pointer for the next non-zero slot; swap every non-zero element into place as you scan. This naturally pushes all zeros to the end while preserving order.

```
function moveZeroes(nums) {
  let insertPos = 0;
  for (let i = 0; i < nums.length; i++) {
    if (nums[i] !== 0) {
      [nums[insertPos], nums[i]] = [nums[i], nums[insertPos]];
      insertPos++;
    }
  }
}
```

Time: $O(n)$ **Space:** $O(1)$

2 Sliding Window & Kadane's

Best Time to Buy and Sell Stock ● Easy

Track the minimum price seen so far and the best profit if you sold today. One pass — no need to check every pair of days.

```
function maxProfit(prices) {
  let minPrice = Infinity;
  let best = 0;
  for (const price of prices) {
    minPrice = Math.min(minPrice, price);
    best = Math.max(best, price - minPrice);
  }
  return best;
}
```

Time: $O(n)$ **Space:** $O(1)$

Maximum Subarray (Kadane's Algorithm) ● Medium

At each index, decide whether to extend the running subarray or start fresh from here — only extend if the running sum is still helping you.

```
function maxSubArray(nums) {
  let maxSum = nums[0];
  let current = nums[0];
  for (let i = 1; i < nums.length; i++) {
    current = Math.max(nums[i], current + nums[i]);
    maxSum = Math.max(maxSum, current);
  }
  return maxSum;
}
```

Time: $O(n)$ **Space:** $O(1)$

Maximum Product Subarray ● Medium

Track both the max AND min product ending at each index — a negative number can flip the smallest running product into the largest one.

```
function maxProduct(nums) {
  let maxProd = nums[0], minProd = nums[0], result = nums[0];
  for (let i = 1; i < nums.length; i++) {
    if (nums[i] < 0) [maxProd, minProd] = [minProd, maxProd];
    maxProd = Math.max(nums[i], maxProd * nums[i]);
    minProd = Math.min(nums[i], minProd * nums[i]);
    result = Math.max(result, maxProd);
  }
  return result;
}
```

Time: $O(n)$ **Space:** $O(1)$

3 Hashing & Prefix Sum

Two Sum ● Easy

Store each number's index in a hashmap as you scan. For every element, check if its complement (target minus the element) was already seen.

```
function twoSum(nums, target) {
  const seen = new Map();
  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];
    if (seen.has(complement)) return [seen.get(complement), i];
    seen.set(nums[i], i);
  }
  return [];
}
```

Time: $O(n)$ **Space:** $O(n)$

Contains Duplicate ● Easy

A Set only keeps unique values — if its size is smaller than the array's length, a duplicate must exist.

```
function containsDuplicate(nums) {
  return new Set(nums).size !== nums.length;
}
```

Time: $O(n)$ **Space:** $O(n)$

Product of Array Except Self ● Medium

Build a running prefix product left-to-right into the result array, then sweep right-to-left multiplying in a running suffix product. No division required.

```
function productExceptSelf(nums) {
  const n = nums.length;
  const result = new Array(n).fill(1);
  let prefix = 1;
  for (let i = 0; i < n; i++) {
    result[i] = prefix;
    prefix *= nums[i];
  }
  let suffix = 1;
  for (let i = n - 1; i >= 0; i--) {
    result[i] *= suffix;
    suffix *= nums[i];
  }
  return result;
}
```

Time: $O(n)$ **Space:** $O(1)$ extra (excl. output)

Subarray Sum Equals K ● Medium

Track running prefix sums in a hashmap of counts. If (currentSum - k) was seen before, every occurrence marks a subarray ending here that sums to k.

```
function subarraySum(nums, k) {
  const seenSums = new Map([[0, 1]]);
  let sum = 0, count = 0;
  for (const num of nums) {
    sum += num;
    if (seenSums.has(sum - k)) count += seenSums.get(sum - k);
    seenSums.set(sum, (seenSums.get(sum) || 0) + 1);
  }
  return count;
}
```

Time: $O(n)$ **Space:** $O(n)$

Longest Consecutive Sequence ● Medium

Put every number in a Set, then only start counting a run from numbers that have no predecessor (num - 1) in the set — that guarantees each run is counted once.

```
function longestConsecutive(nums) {
  const set = new Set(nums);
  let longest = 0;
  for (const num of set) {
    if (!set.has(num - 1)) {
      let length = 1;
      while (set.has(num + length)) length++;
      longest = Math.max(longest, length);
    }
  }
  return longest;
}
```

Time: $O(n)$ **Space:** $O(n)$

4 Sorting & In-Place Rearrangement

Sort Colors (Dutch National Flag) ● Medium

Three pointers — low, mid, high — partition the array into 0s, 1s, and 2s in a single pass. No counting pass or extra array needed.

```
function sortColors(nums) {
  let low = 0, mid = 0, high = nums.length - 1;
  while (mid <= high) {
    if (nums[mid] === 0) {
      [nums[low], nums[mid]] = [nums[mid], nums[low]];
      low++; mid++;
    } else if (nums[mid] === 1) {
      mid++;
    } else {
      [nums[mid], nums[high]] = [nums[high], nums[mid]];
      high--;
    }
  }
}
```

Time: $O(n)$ **Space:** $O(1)$

Majority Element ● Easy

Boyer-Moore voting: increment a counter for the current candidate, decrement for anything else, and swap candidates when the count hits zero. The true majority always survives.

```
function majorityElement(nums) {
  let count = 0, candidate = null;
  for (const num of nums) {
    if (count === 0) candidate = num;
    count += num === candidate ? 1 : -1;
  }
  return candidate;
}
```

Time: $O(n)$ **Space:** $O(1)$

Next Permutation ● Medium

Find the rightmost ascending pair, swap it with the smallest value larger than it to its right, then reverse everything after that point to get the next lexicographic arrangement.

```
function nextPermutation(nums) {
  let i = nums.length - 2;
  while (i >= 0 && nums[i] >= nums[i + 1]) i--;
  if (i >= 0) {
    let j = nums.length - 1;
    while (nums[j] <= nums[i]) j--;
    [nums[i], nums[j]] = [nums[j], nums[i]];
  }
  let left = i + 1, right = nums.length - 1;
  while (left < right) {
    [nums[left], nums[right]] = [nums[right], nums[left]];
    left++; right--;
  }
}
```

Time: $O(n)$ **Space:** $O(1)$

Kth Largest Element in an Array ● Medium

Quickselect partitions around a pivot like quicksort, but only recurses into the one side that contains the target index instead of both — that's what drops it to average $O(n)$.

```
function findKthLargest(nums, k) {
  const targetIdx = nums.length - k;
  function quickSelect(left, right) {
    const pivot = nums[right];
    let p = left;
    for (let i = left; i < right; i++) {
      if (nums[i] <= pivot) {
        [nums[i], nums[p]] = [nums[p], nums[i]];
        p++;
      }
    }
    [nums[p], nums[right]] = [nums[right], nums[p]];
    if (p === targetIdx) return nums[p];
    return p < targetIdx ? quickSelect(p + 1, right) : quickSelect(left, p - 1);
  }
  return quickSelect(0, nums.length - 1);
}
```

Time: $O(n)$ avg, $O(n^2)$ worst **Space:** $O(1)$

5 Binary Search on Arrays

Search in Rotated Sorted Array ● Medium

One half of the array relative to mid is always properly sorted. Check which half that is first, then decide whether the target falls inside that sorted half.

```
function search(nums, target) {
  let left = 0, right = nums.length - 1;
  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (nums[mid] === target) return mid;
    if (nums[left] <= nums[mid]) {
      if (nums[left] <= target && target < nums[mid]) right = mid - 1;
      else left = mid + 1;
    } else {
      if (nums[mid] < target && target <= nums[right]) left = mid + 1;
      else right = mid - 1;
    }
  }
  return -1;
}
```

Time: $O(\log n)$ **Space:** $O(1)$

Find Minimum in Rotated Sorted Array ● Medium

Compare the middle element to the right edge. If mid is greater than the right edge, the minimum must be to the right; otherwise it's at mid or further left.

```
function findMin(nums) {
  let left = 0, right = nums.length - 1;
  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (nums[mid] > nums[right]) left = mid + 1;
    else right = mid;
  }
  return nums[left];
}
```

Time: $O(\log n)$ **Space:** $O(1)$

6 Matrix / 2D Array

Rotate Image ● Medium

Transpose the matrix (flip across the main diagonal), then reverse each row. That combination is equivalent to a 90° clockwise rotation, done fully in place.

```
function rotate(matrix) {
  const n = matrix.length;
  for (let i = 0; i < n; i++) {
    for (let j = i; j < n; j++) {
      [matrix[i][j], matrix[j][i]] = [matrix[j][i], matrix[i][j]];
    }
  }
  for (let i = 0; i < n; i++) {
    matrix[i].reverse();
  }
}
```

Time: $O(n^2)$ **Space:** $O(1)$

Spiral Matrix ● Medium

Track four shrinking boundaries — top, bottom, left, right — and walk each edge in order (right, down, left, up), moving the matching boundary inward after each edge.

```
function spiralOrder(matrix) {
  const result = [];
  let top = 0, bottom = matrix.length - 1;
  let left = 0, right = matrix[0].length - 1;
  while (top <= bottom && left <= right) {
    for (let j = left; j <= right; j++) result.push(matrix[top][j]);
    top++;
    for (let i = top; i <= bottom; i++) result.push(matrix[i][right]);
    right--;
    if (top <= bottom) {
      for (let j = right; j >= left; j--) result.push(matrix[bottom][j]);
      bottom--;
    }
    if (left <= right) {
      for (let i = bottom; i >= top; i--) result.push(matrix[i][left]);
      left++;
    }
  }
  return result;
}
```

Time: $O(m \cdot n)$ **Space:** $O(1)$ extra (excl. output)

Set Matrix Zeroes ● Medium

Use the matrix's own first row and first column as marker space for which rows/columns must become zero, avoiding a separate set or array. Record the two corner flags first.

```
function setZeroes(matrix) {
  const m = matrix.length, n = matrix[0].length;
  let firstRowZero = false, firstColZero = false;
  for (let i = 0; i < m; i++) if (matrix[i][0] === 0) firstColZero = true;
  for (let j = 0; j < n; j++) if (matrix[0][j] === 0) firstRowZero = true;

  for (let i = 1; i < m; i++) {
    for (let j = 1; j < n; j++) {
      if (matrix[i][j] === 0) {
        matrix[i][0] = 0;
        matrix[0][j] = 0;
      }
    }
  }
  for (let i = 1; i < m; i++) {
    for (let j = 1; j < n; j++) {
      if (matrix[i][0] === 0 || matrix[0][j] === 0) matrix[i][j] = 0;
    }
  }
  if (firstRowZero) for (let j = 0; j < n; j++) matrix[0][j] = 0;
  if (firstColZero) for (let i = 0; i < m; i++) matrix[i][0] = 0;
}
```

Time: $O(m \cdot n)$ **Space:** $O(1)$

7 Intervals

Merge Intervals ● Medium

Sort by start time, then walk through once — if the next interval overlaps the last merged one, extend it; otherwise start a new merged interval.

```
function merge(intervals) {
  intervals.sort((a, b) => a[0] - b[0]);
  const result = [intervals[0]];
  for (let i = 1; i < intervals.length; i++) {
    const last = result[result.length - 1];
    if (intervals[i][0] <= last[1]) {
      last[1] = Math.max(last[1], intervals[i][1]);
    } else {
      result.push(intervals[i]);
    }
  }
  return result;
}
```

Time: $O(n \log n)$ **Space:** $O(n)$

Insert Interval ● Medium

Three phases in one pass: copy intervals ending before the new one starts, merge every overlapping interval into the new one, then copy the rest untouched.

```
function insert(intervals, newInterval) {
  const result = [];
  let i = 0;
  while (i < intervals.length && intervals[i][1] < newInterval[0]) {
    result.push(intervals[i++]);
  }
  while (i < intervals.length && intervals[i][0] <= newInterval[1]) {
    newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
    newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
    i++;
  }
  result.push(newInterval);
  while (i < intervals.length) result.push(intervals[i++]);
  return result;
}
```

Time: $O(n)$ **Space:** $O(n)$

Non-overlapping Intervals ● Medium

Sort by end time and greedily keep the interval that finishes earliest — it leaves the most room for everything that comes after, minimizing removals.

```
function eraseOverlapIntervals(intervals) {
  intervals.sort((a, b) => a[1] - b[1]);
  let removals = 0, end = -Infinity;
  for (const [start, finish] of intervals) {
    if (start >= end) end = finish;
    else removals++;
  }
  return removals;
}
```

Time: $O(n \log n)$ **Space:** $O(1)$

Quick Reference — All Problems

#	Problem	Pattern	Time	Space
1	3Sum	Two Pointers	$O(n^2)$	$O(1)$ extra (excl. sort)
2	Container With Most Water	Two Pointers	$O(n)$	$O(1)$
3	Trapping Rain Water	Two Pointers	$O(n)$	$O(1)$
4	Move Zeroes	Two Pointers	$O(n)$	$O(1)$
5	Best Time to Buy and Sell Stock	Sliding Window & Kadane's	$O(n)$	$O(1)$
6	Maximum Subarray (Kadane's Algorithm)	Sliding Window & Kadane's	$O(n)$	$O(1)$
7	Maximum Product Subarray	Sliding Window & Kadane's	$O(n)$	$O(1)$
8	Two Sum	Hashing & Prefix Sum	$O(n)$	$O(n)$
9	Contains Duplicate	Hashing & Prefix Sum	$O(n)$	$O(n)$
10	Product of Array Except Self	Hashing & Prefix Sum	$O(n)$	$O(1)$ extra (excl. output)
11	Subarray Sum Equals K	Hashing & Prefix Sum	$O(n)$	$O(n)$
12	Longest Consecutive Sequence	Hashing & Prefix Sum	$O(n)$	$O(n)$
13	Sort Colors (Dutch National Flag)	Sorting & In-Place Rearrangement	$O(n)$	$O(1)$
14	Majority Element	Sorting & In-Place Rearrangement	$O(n)$	$O(1)$
15	Next Permutation	Sorting & In-Place Rearrangement	$O(n)$	$O(1)$
16	Kth Largest Element in an Array	Sorting & In-Place Rearrangement	$O(n)$ avg, $O(n^2)$ worst	$O(1)$
17	Search in Rotated Sorted Array	Binary Search on Arrays	$O(\log n)$	$O(1)$
18	Find Minimum in Rotated Sorted Array	Binary Search on Arrays	$O(\log n)$	$O(1)$
19	Rotate Image	Matrix / 2D Array	$O(n^2)$	$O(1)$
20	Spiral Matrix	Matrix / 2D Array	$O(m \cdot n)$	$O(1)$ extra (excl. output)
21	Set Matrix Zeroes	Matrix / 2D Array	$O(m \cdot n)$	$O(1)$
22	Merge Intervals	Intervals	$O(n \log n)$	$O(n)$
23	Insert Interval	Intervals	$O(n)$	$O(n)$
24	Non-overlapping Intervals	Intervals	$O(n \log n)$	$O(1)$

Before You Walk In

- Clarify before coding: can the array be empty? Are there duplicates or negative numbers? Is it already sorted?
- State the brute-force approach out loud first, then explain the optimization — interviewers weigh your reasoning as much as the final code.
- Dry-run your solution on a small example (3–5 elements) before declaring it done — it catches off-by-one bugs immediately.
- Say the time and space complexity out loud unprompted; don't wait to be asked.
- If you're stuck, reach for the pattern table on page 1 — the input shape (sorted? contiguous? pairs?) almost always points to the right technique.